

INTERNAL TOOL — INTRODUCTION

FlashTool

ONE ROM FLASH TOOL — CROSS-PLATFORM, MULTI-MODEL

CURRENT PROBLEMS

Why FlashTool?

01 / PLATFORM

Windows only

The flash tool provided by Sharp currently **runs on Windows only**. Linux machines can't **install it** — QA/dev on Ubuntu must borrow a Windows machine or run commands by hand.

02 / FRAGMENTATION

One app per model

Each device model (PS11, E11, E10, E9, G6...) ships with **its own separate flash app**. Users must install and switch between many different tools.

03 / WORKFLOW

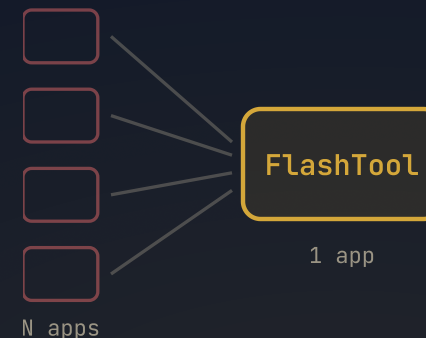
Complex Jenkins flash

Flashing a Jenkins build requires **downloading both the Jenkins ROM and the Official ROM**, then **copying Jenkins files over the Official package** before flashing — manual and error-prone.

THE SOLUTION

One app for everything

- **Cross-platform:** runs on both **Windows and Ubuntu/Linux** — shipped as prebuilt executables, no Python install needed
- **A single app** for every model: G6 / X6 / X5 / X5P, PS10, PS11, E11, E10, E9
- **Modern GUI** (CustomTkinter, dark theme) replacing terminals and scattered tools
- **Direct Jenkins flash:** just pick the Jenkins ROM folder — the tool auto-detects Official/Jenkins, no manual copy-over



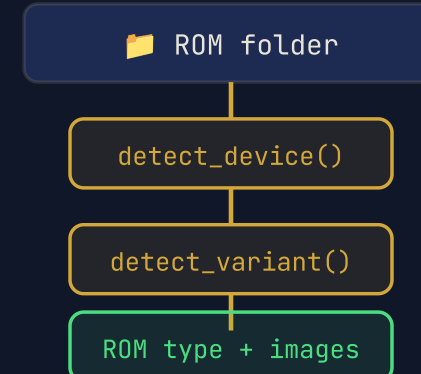
Supported devices

MODEL	FLASH MECHANISM	VARIANTS / NOTES
G6 / X6 / X5 / X5P	Python profile — step-by-step adb/fastboot flow	Auto-resolve <code>system</code> , <code>product</code> , <code>system_ext</code> , <code>vbmeta</code> , <code>super</code> ; regional variant support (ML2...)
PS11	Bash script <code>flash_ps11.sh</code> — 5 phases	Sharp Aquos KIRA; variants: <code>kira</code> , <code>mn4</code> , <code>pdn4</code> , <code>pen4</code> , <code>phn4</code> , <code>tan4</code> , <code>ten4</code>
PS10	Bash script <code>flash_ps10.sh</code> — 5 phases	Variants: <code>mn3</code> , <code>pdn3</code> , <code>pen3</code> , <code>phn3</code> , <code>tan3</code> , <code>tdn3</code> , <code>ten3</code>
E11	Bash script <code>flash_e11.sh</code> — 7 phases	Auto-detects Official vs Jenkins ROM; variants: <code>MC6</code> , <code>PDC6</code> , <code>PEC6</code> , <code>PHC6</code> , <code>PKC6</code>
E10	Bash script <code>flash_e10.sh</code> — 7 phases	Variants: <code>MC5</code> , <code>PDC5</code> , <code>PEC5</code> , <code>PHC5</code> , <code>PKC5</code> , <code>TAC5</code> , <code>TDC5</code> , <code>TEC5</code>
E9	Bash script <code>flash_e9.sh</code> — 7 phases	NAZE I7-OS; variants: <code>MC4</code> , <code>PDC4</code> , <code>PEC4</code> , <code>PHC4</code> , <code>PKC4</code> , <code>TAC4</code> , <code>TDC4</code> , <code>TEC4</code>

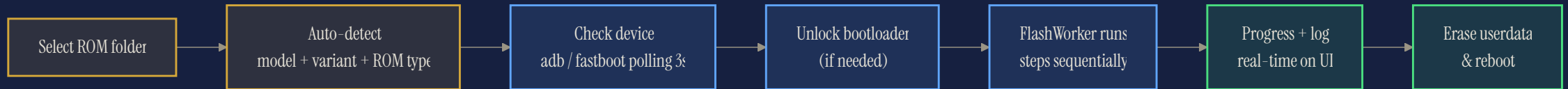
AUTOMATION

Pick a folder — the tool does the rest

- **Auto-detect model** from ROM folder contents: signature files + variant directories (scored, best match wins)
- **Auto-detect variant** (MC6, Kira, ML2...) from subdirectories or `system_ext-*` filenames
- **Auto-detect ROM type** Official / Jenkins — or override manually via dropdown
- **Auto-resolve images** per partition: `system.img`, `product-*.img`, `vbmeta*.img`... no manual file picking



End-to-end flash flow



G6 / X6 / X5 / X5P

13-step adb / fastboot

- ① **Unlock** bootloader + verify
- ② Flash **vbmeta** (disable verification)
- ③ Reboot → wait for ADB → enter **fastbootd**
- ④ Flash **system** / **product** / **system_ext**
- ⑤ Erase **metadata** + **userdata**, reboot

PS11 / E11 / E10 / E9

Script + phase tracking

- ① Validate package & enter bootloader
- ② Flash bootstrap → enter **fastbootd**
- ③ Flash dynamic partitions (super)
- ④ Flash bootloader / modem / vbmeta
- ⑤ Set active slot, wipe, reboot

Each script phase maps to its own step card on the UI — you can see exactly where the script is.

EXPERIENCE & SAFETY

Full control over the process

- **3s device polling:** adb / fastboot / disconnected status shown in real time
- **Bootloader helper:** reboot to bootloader, check unlock status (`ro.boot.flash.locked`, `getvar unlocked`), separate confirmed unlock button
- **Per-step progress:** progress bar + elapsed time, parses sparse-image upload from fastboot
- **Live console log** + **Cancel** at any step
- **Skip Setup Wizard:** mark provisioned via ADB and reboot — bypass first-run setup screens



DISTRIBUTION & UPDATES

Automated builds, self-updating

2

PLATFORMS

FlashTool-Linux +
FlashTool-Windows.exe via
PyInstaller

CI/CD

GITHUB ACTIONS

Push tag v* → auto build &
publish to Releases

Auto

SELF-UPDATE

Checks GitHub Releases,
downloads & replaces
binary, restarts app

v1.3.3

CURRENT VERSION

Python 3.10+ /
CustomTkinter / adb +
fastboot

SUMMARY

One tool — every model — every platform

- Linux now flashes like Windows — no more dependence on a Windows machine
- 9 models in 1 app — no more per-model tool fragmentation
- Jenkins ROMs flash directly — the complex Official copy-over workflow is gone

FLASHTOOL V1.3.3 — INTERNAL TOOL